B. Algorithm Design and Program Decomposition

1. Principal Computational Algorithm

The Population Simulator will be a Universe 25 generation-based population simulation algorithm to simulate the mouse population changes over time. The algorithm starts with the number of males and females and resources at generation 0. The program determines births, deaths, resource use and replenishment for each subsequent generation, using the user inputted parameters.

The model assumes that a male and a female are necessary for reproduction and that the number of births will be related to the resources and the rate of births between the male and female. The number of deaths is determined by the base death rate, and more deaths will happen if there are not enough resources. Each mouse that survives and each newborn mouse will eat one resource unit each generation. The number of births and baseline deaths will be rounded to the nearest whole number.

The program will update the male and female populations after each generation, compute the remaining resources, and report on the results after each update. This is repeated for a number of generations until the number of generations mentioned is completed.

2. Main Algorithm Pseudocode

START

1. Enter the initial male and female populations, initial resources, number of generations, birth rate, death rate, and resource replenishment.
2. Ensure that all inputs are valid and invalid input is rejected.
3. Record the initial population and resources as Generation 0.
4. For each generation:
    - Add the configured resource replenishment.

    - Determine normal mortality rates for males and females.
    - Remove deaths from the existing population.
    - Examine the availability of resources for the survival of mice.
    - If resources are low, apply extra deaths, depending on the resource shortage.
    - Determine if there are at least one male and one female still alive.
    - Estimate number of births from surviving females and birth rate.
    - Plan births within resources.
    - Separate male and female newborns.
    - Subtract the food used by mice that lived and newborn mice.
    - Determine the new number of the population.
    - Count the males, females, births, deaths, and other resources.
5. Repeat as many times as the desired generation limit.
6. Show the results table on generation by generation.
7. Create population and resources change charts.
8. Show the final simulation summary.

END

The algorithm must maintain the following conditions:

- New population = Previous population + Births − Deaths
- Total population = Male population + Female population
- Population and resource values must never be negative.
- Births cannot exceed the number supported by available resources.

3. **Major Program Components and Responsibilities**

Component	Responsibility
Input Validation	Checks user inputs and prevents invalid settings.

Population Simulation	Calculates births, deaths, resource limitations, and population changes.
Resource Management	Handles resource consumption, replenishment, and shortages.
Simulation Controls	Manages the Run, Step, and Reset operations.
Data Reporting	Records and displays the results for each generation.
Data Visualization	Generates charts showing population growth and resource changes
Graphical User Interface	Allows users to enter parameters, control the simulation, and view results.

4. Proposed Python Functions

The following functions are proposed for the application:

Function	Purpose
validate_inputs()	Checks whether all input values are valid.
initialize_state()	Creates the starting simulation state and records Generation 0.
calculate_deaths()	Calculates baseline deaths based on the death rate.
apply_resource_shortage()	Adjusts the surviving population when resources are insufficient.

calculate_births()	Calculates possible births based on reproduction rules and resources.
advance_generation()	Updates the population and resources for one generation.
run_simulation()	Executes the simulation until the selected generation limit.
step_simulation()	Advances the simulation by one generation.
reset_simulation()	Restores the original settings and simulation state.
create_record()	Stores the results of each generation.
update_table()	Displays generation-by-generation data.
update_charts()	Updates population and resource graphs.
summarize_end_state()	Produces the final simulation summary.

5. Interaction Between Functions

The functions will be called in an ordered sequence to simulate the desired behaviour. First the program saves the user's settings and calls validate_inputs() to validate those inputs. When the inputs are accepted, initialize_state() sets up the initial simulation state.

Advance_generation() is called by the user when the Run or Step button is pressed to process the next generation, which means calling the functions that calculate deaths, resource scarcity and births. The new values of population and resources are then stored with create_record().

The results are updated and the latest are displayed after each generation with update_table() and update_charts(). The Run function is executed until the desired number of

generations has been reached, and the Step function advances a single generation at a time. The Reset function resets the simulation.

The division of the application into smaller tasks allows the group to build and test each individual task without worrying about the consistency of the simulation and the production of accurate results based on the model assumed by the simulation.

AI Assistance Disclosure: ChatGPT and Quillbot was used to assist in preparing the algorithm explanation. The group will review and test the algorithm before implementing the final application.